StudyBuddy

Knowledge Goal Demonstration

[KG1] Endpoint Definitions

File Reference: app/main.py:323-337
Code Fragment:

Python

```python
# API Endpoint 3 (KG1): Single user get by id.
@app.get("/api/users/{user_id}")
def api_get_user(
   user_id: int,
   session: Session = Depends(get_session)
):
   user = user_service.find_user_by_id(session, user_id)

   if not user:
       raise HTTPException(
           status_code=status.HTTP_404_NOT_FOUND,
           detail="User not found"
       )

   return user
```

Justification: The GET /api/users/{user_id} endpoint retrieves data for a specific user. The {user_id} segment of the URL is a path parameter used by the server to identify the requested user resource.

[KG2] HTTP Methods & Status Codes

File Reference: app/main.py:357-372
Code Fragment:

Python

```python
# API Endpoint 5 (KG2): Delete user.
@app.delete("/api/users/{user_id}", status_code=status.HTTP_204_NO_CONTENT)
def api_delete_user(
```


Naina Singh, nainasingh@umass.edu

```
    user_id: int,
    session: Session = Depends(get_session)
):
    user = user_service.find_user_by_id(session, user_id)

    if not user:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User not found"
        )

    user_service.delete_user(session, user)
    return None
```

Justification: The DELETE /api/users/{user_id} endpoint removes a specific user resource. A successful deletion returns a 204 NO CONTENT status code, while a 404 NOT FOUND error is raised if the user does not exist.

[KG3] Endpoint Validation

File Reference: app/models.py:44-51
Code Fragment:

Python

```python
# KG3: Request/Response structure for creating a new user with validation
class UserCreate(SQLModel):
    name: str = Field(min_length=1, max_length=50)
    password: str = Field(min_length=6)
    email: Optional[EmailStr] = None
    year: Optional[str] = None
    style: Optional[str] = None
    availability: Optional[str] = None
```

Justification: The Pydantic structure validates incoming data before it reaches the application logic. Field constraints ensure names are between 1 and 50 characters and passwords are at least 6 characters long. FastAPI automatically rejects invalid requests with a 422 Unprocessable Entity error.

Naina Singh, nainasingh@umass.edu

StudyBuddy

[KG4] Dependency Injection

File Reference: app/database.py:7-10
Code Fragment:

```python
def get_session():
    with Session(engine) as session:
        yield session
```

Used in: app/main.py:305-310

```python
# API Endpoint 1 (KG7,KG4): Creates new user.
@app.post("/api/users", status_code=status.HTTP_201_CREATED)
def api_create_user(
    user_data: UserCreate,
    session: Session = Depends(get_session)
):
```

Justification: The Depends(get_session) decorator injects the database session into the route function externally. This improves code organization by separating the database connection management from the core logic, making the code easier to test and maintain.

[KG5] Data Model

File Reference: app/models.py:7-16
Code Fragment:

```python
# Link table for many-to-many relationship between Users and Courses
class UserCourseLink(SQLModel, table=True):
    user_id: Optional[int] = Field(default=None, foreign_key="user.id",
primary_key=True)
    course_id: Optional[int] = Field(default=None, foreign_key="course.id",
primary_key=True)


# Database Course model with validation
class Course(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
```


Naina Singh, nainasingh@umass.edu

```
    code: str = Field(min_length=3, max_length=15, index=True)
    users: List["User"] = Relationship(back_populates="courses",
link_model=UserCourseLink)
```

Justification: This is the formal data structure for how courses are organized and stored in the database. It includes a link table for the many-to-many relationship between users and courses, with proper foreign keys and relationships that reflect the database structure.

[KG6] CRUD Operations & Persistent Data

File Reference: app/main.py:317-321
Code Fragment:

Python
```
# API Endpoint 2 (KG7,KG6): All users as JSON
@app.get("/api/users")
def api_list_users(session: Session = Depends(get_session)):
    users = session.exec(select(User)).all()
    return users
```

Justification: This performs a Read operation on the PostgreSQL database to retrieve all users. Because the data is stored in the database, it is persistent and remains even after the server restarts. This is one of the four essential CRUD operations supported by the application.

[KG7] API Endpoints & JSON

File Reference: app/main.py:305-315
Code Fragment:

Python
```
# API Endpoint 1 (KG7,KG4): Creates new user.
@app.post("/api/users", status_code=status.HTTP_201_CREATED)
def api_create_user(
    user_data: UserCreate,
    session: Session = Depends(get_session)
):
    try:
        user = user_service.create_user(session, user_data)
```


Naina Singh, nainasingh@umass.edu

```
        return user
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

Justification: This API endpoint for machine-to-machine communication. It accepts input formatted as JSON (validated by the UserCreate) and returns a JSON object in response. This makes the endpoint consumable by other applications, rather than for direct use in a browser.

[KG8] UI Endpoints & HTMX

File Reference: app/main.py:153-172
Code Fragment:

Python

```python
# UI Endpoint 9 (KG8): Adds course to user profile.
@app.post("/add_course")
def add_course(
    request: Request,
    new_course: str = Form(""),
    session: Session = Depends(get_session)
):
    u = current_user(session, request)
    if not u:
        return RedirectResponse("/", status_code=status.HTTP_303_SEE_OTHER)

    if new_course.strip():
        course_service.add_course_to_user(session, u, new_course.strip())

    session.refresh(u)
    # Partial update for HTMX
    if request.headers.get("HX-Request") == "true":
        return render_course_list_partial(request, u)

    return RedirectResponse("/profile", status_code=status.HTTP_303_SEE_OTHER)
```

Template: app/templates/profile.html:57-60

Naina Singh, nainasingh@umass.edu

```HTML
<form
     method="post"
     hx-post="/add_course"
     hx-target="#courses-list"
     hx-swap="outerHTML">
   <input name="new_course" placeholder="Exact course code (e.g., COMPSCI
326)">
   <button type="submit">Add course</button>
 </form>
```

Justification: This endpoint detects HTMX requests by checking the HX-Request header and returns partial HTML instead of a full page. The HTMX attributes in the template make a request that updates only the course list div, giving a smooth user experience.

[KG9] User Interaction (CRUD)

File Reference: app/main.py:54-85
Code Fragment:

```Python
# UI Endpoint 3 (KG9): Creates new user account
@app.post("/create", response_class=HTMLResponse)
def create_post(
   request: Request,
   name: str = Form(...),
   password: str = Form(...),
   email: str = Form(""),
   year: str = Form(""),
   style: str = Form(""),
   availability: str = Form(""),
   course: str = Form(""),
   session: Session = Depends(get_session),
):
   try:
       user_data = UserCreate(
           name=name.strip(),
```


Naina Singh, nainasingh@umass.edu

```
            password=password,
            email=email.strip() if email else None,
            year=year,
            style=style,
            availability=availability
        )
        user = user_service.create_user(session, user_data)
        if course.strip():
            course_service.add_course_to_user(session, user, course.strip())
        return templates.TemplateResponse("account_created.html", {
            "request": request,
            "name": user.name
        })

    except ValueError as e:
        return HTMLResponse(str(e), status_code=400)
```

Justification: This shows a Create operation triggered by user interaction through a web form. When a user clicks "Create Account," the data flows from the UI through this endpoint to the backend CRUD operation, which persists a new user in the database.

[KG10] Separation of Concerns

File Structure:

```
None
StudyBuddy/
├── app/
│   ├── main.py              // FastAPI routes - Presentation Layer
│   ├── models.py            // SQLModel data models - Data Model Layer
│   ├── database.py          // Database - Data Access Layer
│   ├── services/
│   │   ├── users.py         // User logic - Business logic layer
│   │   ├── courses.py       // Course logic - Business logic layer
│   │   └── matching.py      // Partner matching logic - Business logic layer
```

```
|   └── templates/           // Jinja2 HTML templates
|       ├── home.html
|       ├── create_account.html
|       ├── login.html
|       ├── profile.html
|       ├── course_list.html
|       ├── partners.html
|       ├── incoming.html
|       ├── matches.html
|       └── account_created.html
├── static/
|   └── styles.css
├── requirements.txt         // Dependencies
├── README.md
└── KGD.pdf                  // Knowledge Goal Demonstration document
```

Justification: The application separates concerns into distinct layers. The presentation layer (main.py) handles HTTP requests and responses, the business logic layer (services/) contains the application's core functionality, and the data access layer (database.py) manages database connections. This design makes the code easier to understand, test, and maintain.

Naina Singh, nainasingh@umass.edu